
accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000



500 Optimization Overview:
You Can't Optimize Too Soon*

Bob Ebert
Consulting Engineer, Palm Inc.

PalmSource
Conference 2000



500 Optimization Overview:
You Can't Optimize Too Soon*

*This is an example of a
PROFOUND TRUTH

PalmSource
Conference 2000

'Op • ti • "mize

1. to make as effective, perfect, or useful as possible.
2. to write or rewrite (the instructions in a computer program) for maximum efficiency and speed in retrieval, storage, or execution.
3. Math. to determine the maximum or minimum values of (a specified function that is subject to a set of constraints).
4. to make the best of.
5. to be optimistic.

'Op • ti • "mize

1. **to make as effective, perfect, or useful as possible.**
2. to write or rewrite (the instructions in a computer program) for maximum efficiency and speed in retrieval, storage, or execution.
3. Math. to determine the maximum or minimum values of (a specified function that is subject to a set of constraints).
4. to make the best of.
5. to be optimistic.

Effective, Perfect, Useful

- "*La perfection est atteinte non quand il ne reste rien à ajouter, mais quand il ne reste rien à enlever.*"

  Antoine de Saint Exupery (1900-1944)



Effective, Perfect, Useful

- *"Perfection is attained not when there is no longer anything to add, but when there is no longer anything to take away."*

  Antoine de Saint Exupery (1900-1944)

- *"La perfection est atteinte non quand il ne reste rien à ajouter, mais quand il ne reste rien à enlever."*

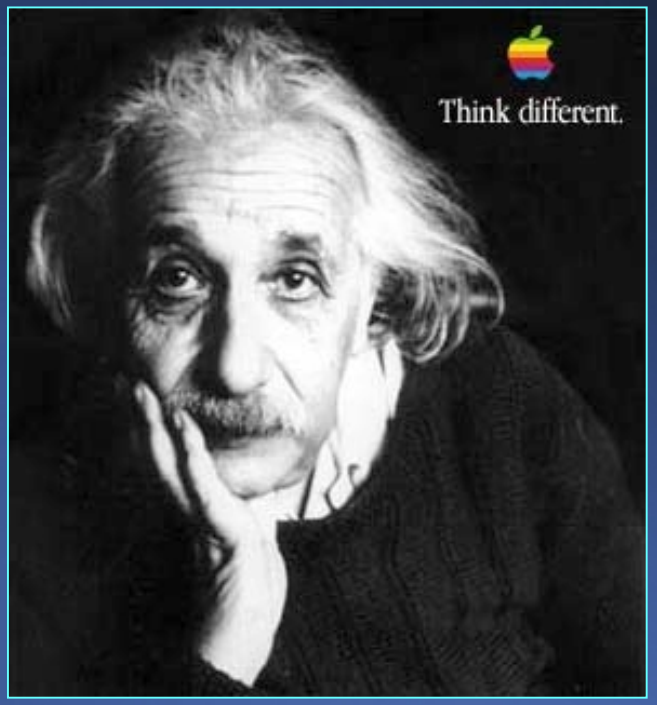


PalmSource Conference 2000

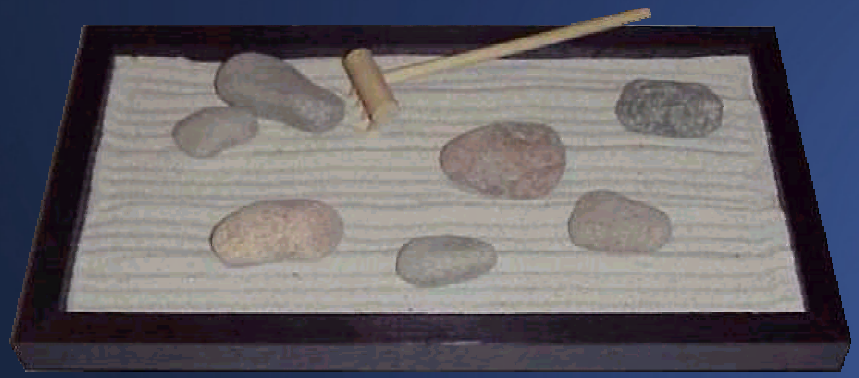
…As Possible

- *"Things should be made as simple as possible, but no simpler."*

  Albert Einstein (1879–1955)



Zen of Palm



- PalmSource talks:
  - ’98 by Rob Hitani
  - ’99 by Michael Lunsford
  - ’00 by Michael Mace
- Palm Publication
- Zen in a Nutshell:
  - Pay attention to what your customers are doing

PalmSource Conference 2000

'Op • ti • "mize

1. to make as effective, perfect, or useful as possible.
2. **to write or rewrite (the instructions in a computer program) for maximum efficiency and speed in retrieval, storage, or execution.**
3. Math. to determine the maximum or minimum values of (a specified function that is subject to a set of constraints).
4. to make the best of.
5. to be optimistic.

Performance Optimization

- Speed
  - CPU Cycles
  - Instructions
  - Memory Accesses
- Memory
  - Dynamic Heap
  - Stack
  - User Storage
- Battery

Code Optimization

- Michael Abrash
  Zen Of Assembly Language
  Code Optimization
  Graphics Programming
  - Sadly, Intel specific
  - Many great lessons for M68K too

- You do not know where/why your code is slow
- Measure. Rewrite. Validate.

POSER Profiling

Function Name	Count	Only	%	+Children	%
functions	0	0.000	0.0	385.034	83.8
SysAppLaunch	1	0.133	0.0	322.621	70.2
PrvCallWithNewStack	1	0.010	0.0	301.676	65.7
__Startup__+$1B08	1	0.004	0.0	301.665	65.7
__Startup__+$1B18	1	0.016	0.0	301.661	65.7
PilotMain	1	0.009	0.0	223.680	48.7
AddrPilotMain	1	0.011	0.0	223.671	48.7
unknown $10CA106A	1	0.009	0.0	223.660	48.7
EventLoop	1	0.101	0.0	181.415	39.5
StartApplication	1	0.090	0.0	41.977	9.1
FrmGotoForm	1	0.018	0.0	0.258	0.1
SysAppStartup	1	0.097	0.0	77.965	17.0
UIReset	1	0.070	0.0	72.460	15.8
DmGet1Resource	2	0.058	0.0	3.517	0.8

POSER Profiling, HostControl.h

```
HostProfileInit(long maxCalls,
                long maxDepth)
HostProfileStart(void)
HostProfileStop(void)
HostProfileDump(const char* filename)
HostProfileCleanup(void)

HostProfileDetailFn(void* addr,
                    HostBoolType logDetails)
long HostProfileGetCycles(void)
```

Speed

- Time to read instructions from memory is bounding most operations
  - Minimum 4 cycles for every 16-bit access
  - No instruction cache
  - CPU is fetching the next instruction while executing the current one
- Most often it's not your app, but Palm OS
- Find the most efficient OS routines to use
- Less Code means Faster Applications
- Go to the Optimization Lab for more

Memory

- Limited heap, stack, and “disk” space
- ‘pref’ #0 resource lets you specify the stack size
  - If you need more
  - If you use less (stack comes from dynamic heap)
  - (New) PalmDebugger w/Debug ROM
- Feature Pointers
  - Trade dynamic heap for storage heap
  - For data that needs to stick around a long time, and doesn’t change often
  - Watch out for “out of memory”

Battery

- Easy to forget about this one
- Penalty you pay for sub-optimal code
  - CPU cycles
  - Video refresh
  - Memory refresh
  - Serial/IR port
  - Attached devices
  - Speaker

Beware Compiler Optimizations

- Code with minimal optimizations
- Debug with no optimizations
  - Code is harder to follow
  - Register re-use means variables hard to track
- Verify compiler-optimized code
  - Often larger
  - Sometimes slower
  - Try different settings, always validate
  - Consider optimizing for “size” as well

Optimizing Too Soon Is a Waste of Time

- You don't yet know where things are slow
- Designs change late in the game
- Adds "low level" complexity
  - Code is less portable
  - Harder to maintain
  - Less reliable
- So, do it last



'Op • ti • "mize

1. to make as effective, perfect, or useful as possible.
2. to write or rewrite (the instructions in a computer program) for maximum efficiency and speed in retrieval, storage, or execution.
3. **Math. to determine the maximum or minimum values of (a specified function that is subject to a set of constraints).**
4. to make the best of.
5. to be optimistic.

The Bigger Picture

- The Plan:



- Murphy's Laws:
  - Nothing is as easy as it looks
  - Everything takes longer than you think
  - Whatever can go wrong, will go wrong

*We'll do more for the next release...

PalmSource Conference 2000

The Bigger Picture

- The Plan:

  Concept Design Implement Debug Optimize Ship

- Murphy's Laws:
  - Nothing is as easy as it looks
  - Everything takes longer than you think
  - Whatever can go wrong, will go wrong
- The Result:

  Optimize*

  Concept Design Im-ple-ment-a-tion Debuggggg Ship

- You can't optimize early, so you end up not optimizing at all

*We'll do more for the next release...

PalmSource Conference 2000

'Op • ti • "mize

1. to make as effective, perfect, or useful as possible.
2. to write or rewrite (the instructions in a computer program) for maximum efficiency and speed in retrieval, storage, or execution.
3. Math. to determine the maximum or minimum values of (a specified function that is subject to a set of constraints).
4. **to make the best of.**
5. to be optimistic.

You Can't Start Optimizing Soon Enough

- Don't do detail level optimization during concept, design, coding, or debugging
- Focus on optimizing the concept, design, coding, or the debugging
- "Which is faster, MemPtrUnlock or MemHandleUnlock?" (Bzzzt.)

Starting Soonest: Concept

- "Do I need to write this program at all?"
  - Don't be afraid to say "no"
- "Who will use this (necessary) program?"
  - Give these people names and lives
- "What is the most important thing they will do with it?"
  - Write a one paragraph "executive summary"
  - Put it up where you can see it

Don’t Do It If You Can’t Do It Right

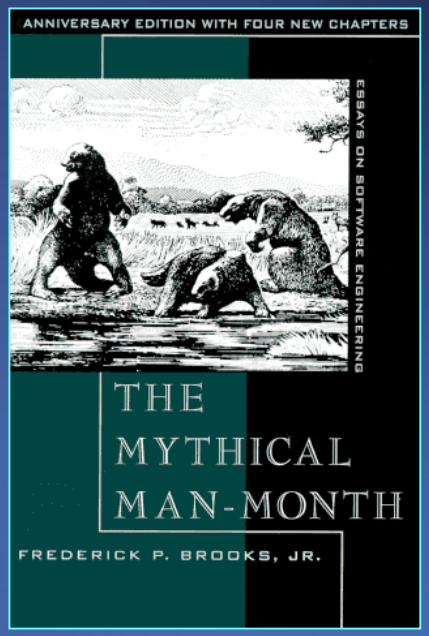
- People will criticize you more for things that are done poorly than for things that are not done at all
- Avoid temptation to “take a shot at it”
- Really think it through, beginning to end
- Or don’t bother doing it

Get the Requirements Right

- This is the hardest part of a software project
- Establish the technical requirements
  - Interfaces to people (users, administrators)
  - Interfaces to machines (APIs, wire formats)
- Nothing cripples a project more than getting this wrong
- Nothing is harder to fix later
- (In fact, this is actually impossible to do)

Optimizing Time to Market

- Frederic P. Brooks Jr.
  The Mythical Man Month: Essays on Software Engineering
- Original 1974
- 2nd edition 1995
- "I believe the hard part of building software to be the specification, design, and testing of this conceptual construct, not the labor of representing it and testing the fidelity of the representation. We still make syntax errors, to be sure; but they are fuzz compared with the conceptual errors in most systems."



Do Not Ignore This Advice

- “A couple of months in the lab can often save a couple of hours in the library.”

Do Not Ignore This Advice

- “A couple of months in the lab can often save a couple of hours in the library.”
- Being part of a late, over-budget, bug-ridden software project* will save you the trouble of reading this book
  - *Actually, it’ll take more than one…

Optimizing Early: Design

- "Always design a thing by considering it in its next larger context—a chair in a room, a room in a house, a house in an environment, an environment in a city plan."

  Eliel Saarinen (1873–1950)







What Design to Optimize?

- Solving the User's Problems
- User Interface
- Application Programmer Interface
- Communications (Bandwidth)
- Programmer Resources
- Physical Resources

Optimizing User Interface

- Roger Flores's talk (this morning)
  - 502: Optimizing the User Interface
  - Get the video…
- Study your target customers
- Take advantage of user's instincts
- Re-use already learned behaviors
  - Stick to standard Palm UI elements
- Keep It Simple, Stupid (Avoid Complexity)
- Minimize Taps

Asymmetry Is Not Bad

- Observation: Engineers love symmetry
- Ordered, conforming, regular, simple, predictable, even
  - For every New, let there be a Delete
- To achieve balance, harmony, and proportion, you often have to break strict symmetry

Beware User Testing

- Note: not “ignore user testing”!
- It’s hard to do well, bad user testing is worse than none
- Stay involved, nothing beats first hand experience
- User testing will only reveal problems, it will not yield solutions
- Graffiti is “too hard” according to many reviews
- User Observation is better
  - If you can find users in their natural habitat...

Optimizing APIs

- Easy to learn?
  - Does it match other existing APIs?
  - Did you get the abstraction right?
- Easy to use?
  - Do you end up doing extra work to use it?
- Portable
- Flexible
- Supportable
- Asymmetry is not always bad

Optimizing Communications (Bandwidth)

- 503: Optimizing Use of the Serial Manager
  - By Bruce Thompson
- How little can you transmit and still make sense?
- Web Clipping vs. WAP
- HotSync
  - Every modified record impacts sync time
  - Watch out for saved user prefs
- InfraRed
  - vCard, vCal aren't very minimal formats
  - XML in general is not very minimal

Optimizing Programmer Resources

- Adding more programmers late does not help
  - Remove promised but not completed features
- Invest in support:
  - Design Tools
  - Coding
  - Debugging Aids
- Make your own tools (if you need to)
  - PalmDebugger
  - Poser Profiler
  - Debug checks, trace support, host control

Three Tips for Coders

- ...or Instincts That Will Steer You Wrong

- Throw the First One Away
- Use Bad Placeholders
- Automate Later

Throw the First One Away

- Prototype
  - Assume you'll get it wrong the first time
- Validate the conceptual structure
  - Don't worry about:
    - Performance
    - Exception Handling
    - Error cases
  - Do worry about:
    - Design
    - Usage patterns
    - Conceptual Model
- You will be tempted to re-use some of it...



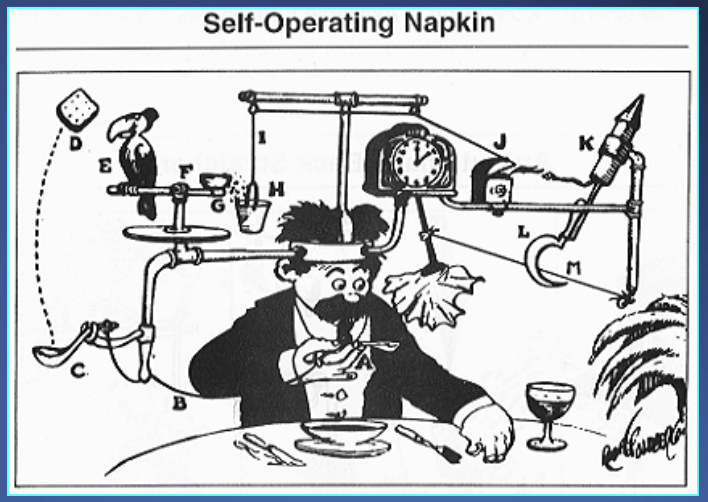
Use Bad Placeholders

- Engineers aren't generally good at art or English
- They don't take the time to reflect or be consistent
- So, make any "placeholders" deliberately bad
- Result: you won't forget to change them later



Automate Later

- Engineers constantly think about automating tasks
  - "We can make a script to do that"
- Sometimes, they even automate a task instead of doing it
- That's like optimizing too soon, you can't really understand a task and how it varies until you've done it a few times, or observed someone doing it a few times
- Automate later



'Op • ti • "mize

1. to make as effective, perfect, or useful as possible.
2. to write or rewrite (the instructions in a computer program) for maximum efficiency and speed in retrieval, storage, or execution.
3. Math. to determine the maximum or minimum values of (a specified function that is subject to a set of constraints).
4. to make the best of.
5. **to be optimistic.**

Don’t Optimize, Just Wait…

- You can ride the “Moore’s Law” curve
  - Gordon Moore, 1965, observed that chip capacity doubles every 18–24 months
- Applies to physical resources:
  - MIPS, Memory, Battery, Bandwidth

Don’t Optimize, Just Wait…

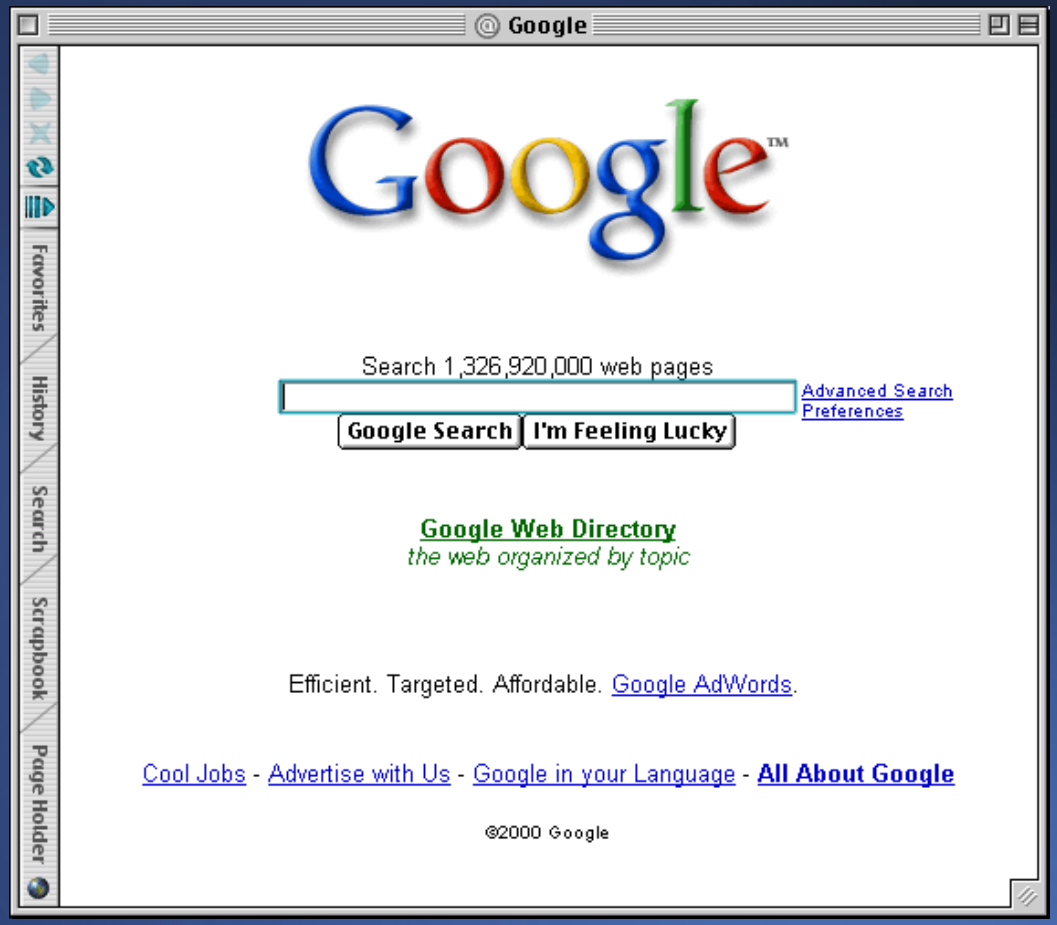
- You can ride the “Moore’s Law” curve
  - Gordon Moore, 1965, observed that chip capacity doubles every 18–24 months
- Applies to physical resources:
  - MIPS, Memory, Battery, Bandwidth
- Does NOT apply to human resources:
  - User IQ, design skills, coding ability, bug fix rate
- Where will the people who didn’t wait be in 18–24 months?

Optimization in the Palm Economy

- Optimal design is better rewarded in the handheld space
- Because the design space is so constrained
  - User's Attention Span
  - Memory, Battery, Screen
  - Cost
- More sensitive to optimization as a holistic development process than other platforms

Not Just the Palm Economy

- These guys get it…



You Can't Optimize Too Soon*

*This is a **Profound Truth**

- The negation of a **SIMPLE TRUTH** is a simple falsehood

  "Two plus two is four."
  "Two plus two is not four."

You Can't Optimize Too Soon*

*This is a **Profound Truth**

- The negation of a **SIMPLE TRUTH** is a simple falsehood

  "Two plus two is four."
  "Two plus two is not four."

- The negation of a **PROFOUND TRUTH** may be another profound truth

  "Handhelds are not like computers."
  "Handhelds are just like computers."

You Can't Optimize Too Soon*

*This is a **Profound Truth**

- You Can't (Optimize Too Soon)
  - You should wait until everything else is done to do the detailed coding optimizations that improve performance
- You Can't Optimize (Too) Soon
  - You should start optimizing as soon as possible
  - But, at the appropriate level:
    - Problem and Requirements at Concept phase
    - UI, API, Algorithms at Design phase
    - Code at the (end of the) Implementation phase

accelerate

collaborate

innovate

communicate



Q&A


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000